MOLE User's Group Meeting

MOLE-OSE Leadership Team[1]

[1]Computational Science Research Center
San Diego State University

Outline

- MOLE Library and Meeting Introduction (Castillo & Pagallo)
- MOLE OSE Sustainability & Governance (Drummond)
- MOLE Library Updates
  - MATLAB/Octave recent updates (Dumett)
  - C++ recent updates (Drummond)
- Example applications
  - GCCOM II
  - Dispersive Waves
- Summary of MOLE peer-review (Paolini)
- Current and Future MOLE Language Implementations (Paolini & Barra)
- MOLE OSE 2026-2027 Strategic Plans (Castillo, Barra & Drummond)
- Open Floor

2026 MOLE Users' Group (MUG) Meeting

Agenda:

- **10:00 - 10:15 AM** Welcome to the 2026 MUG meeting and MOLE Library Introduction
- **10:15 - 10:30 AM** MOLE OSE Sustainability and Governance
- **10:30 - 10:45 AM** MOLE Library Updates
- **10:45 - 11:00 AM** Showcasing 2 MOLE Applications
- **11:00 - 11:15 AM** Summary of MOLE Peer-reviews
- **11:15 - 11:30 AM** Future MOLE Language Implementations
- **11:30 - 11:45 AM** MOLE OSE 2026-2027 Strategic Plans
- **11:45 - 11:55 AM** Open floor
- **11:55 AM - 12:00 PM** Group photograph and Meeting adjournment

Outline

- MOLE Library and Meeting Introduction (Castillo & Pagallo)
- MOLE OSE Sustainability & Governance (Drummond)
- MOLE Library Updates
  - MATLAB/Octave recent updates (Dumett)
  - C++ recent updates (Drummond)
- Example applications
  - GCCOM II
  - Dispersive Waves
- Summary of MOLE peer-review (Paolini)
- Current and Future MOLE Language Implementations (Paolini & Barra)
- MOLE OSE 2026-2027 Strategic Plans (Castillo, Barra & Drummond)
- Open Floor

The MOLE Library and Meeting Introduction

1. Introduction to the MOLE Library
2. MUG's purpose
3. What we learned from Interviews in Summer 2024
4. Invitation to MOLE Tutorial

What is MOLE?

MOLE is the building block to create computationally efficient codes to solve partial differential equations using mimetic difference methods

What are Mimetic Difference Methods?

Mimetic Difference methods are numerical schemes,based on Mimetic Difference Operators, to solve partial differential equations

Mimetic Difference Methods

- Mimetic methods - constructs discrete analogs of div and grad operators
- The discrete operators mimic the properties of their continuum counterparts
- The resulting schemes are more faithful to the physics of the problem
- Operators have uniform high order of accuracy at the boundaries & interiors of grid

What are Mimetic Operators?

- Mimetic operators are discrete analogs of the first order invariant operators divergence, gradient and curl.
- Mimetic Operators satisfy in the discrete sense the same properties that their continuum counterpart's do
- Mimetic methods construct discrete numerical schemes based on mimetic operators like divergence, gradient, curl, Laplacian, etc.

MOLE: Mimetic Operators Library Enhanced

- MOLE is an open-source library that implements high-order mimetic operators.
- It provides discrete analogs of the most common vector calculus operators: divergence, gradient, curl, and Laplacian.
- These operators act on functions discretized over staggered grids (uniform, nonuniform, and curvilinear), and they satisfy local and global conservation laws.

2026 MUG Meeting

Purpose of the MOLE Users' Group (**MUG**) Meeting :

- Foster collaboration among all MOLE users and developers,
- An opportunity for the MOLE OSE Community to come together and provide feedback on the library,
- An opportunity for the MOLE Leadership Team (MOLT) to share updates and current development plans,
- Share information about software engineering governing and onboarding documents

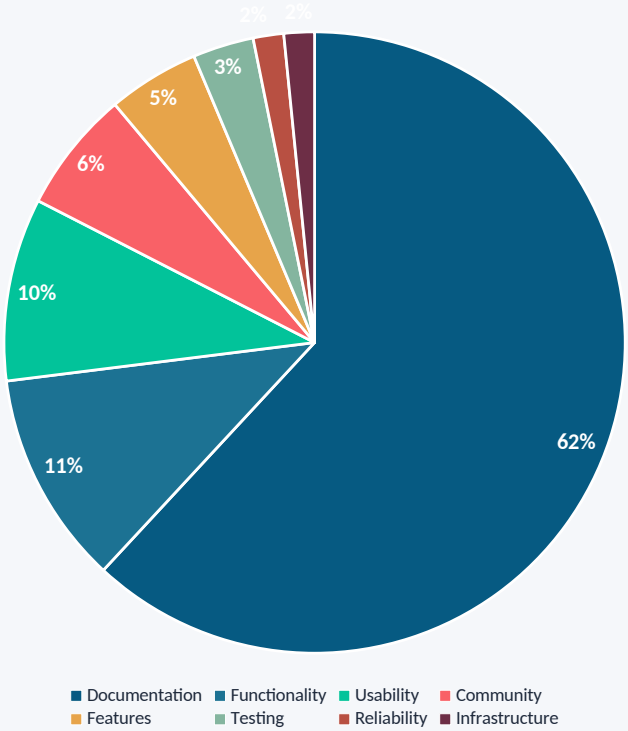
2024 Survey Goals

- Identify high-impact upgrades to MOLE's infrastructure and implementation
- Gather feedback on new features and functionality
- Understand what new users need to adopt MOLE
- Learn best OSE practices from experienced users

2024 Survey Design

- Interviewed 65 participants: 39 users, 26 non-users
- Interviews were conducted in person, by phone or via zoom
- Interviews were transcribed and tabulated

USER Requirements by Category

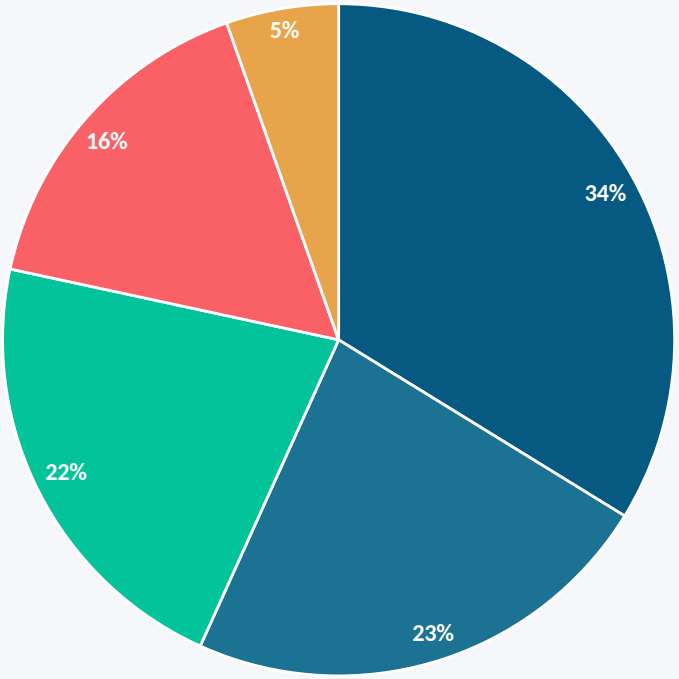


Aggregated from 39 USER respondents | Total items: 63

Category Breakdown

- **Documentation** (39 items, 62%)
  Tutorials, examples, API docs, guides, installation, comments
- **Functionality** (7 items, 11%)
  Error metrics, GPU support, Fortran, fractional PDEs, security
- **Usability** (6 items, 10%)
  Ease of use, readability, portability, adoption steps, bug fix
- **Community** (4 items, 6%)
  Reddit channel, users meetings, developer support, newsletter
- **Features** (3 items, 5%)
  Dirichlet BCs, Hermitian Adjoint, Operator
- **Testing** (2 items, 3%)
  Test cases, unit tests
- **Reliability** (1 item, 2%)
  Comparative analysis
- **Infrastructure** (1 item, 2%)
  Standard build infrastructure

NON USER Requirements by Category



■ Functionality ■ Documentation ■ Community ■ Usability ■ Reliability

Aggregated from 26 NON USER respondents | Total items: 74

Category Breakdown

- **Functionality** (25 items, 34%)
  Performance, GPU support, portability, licensing, Julia, security
- **Documentation** (17 items, 23%)
  Build instructions, tutorials, examples, API, benchmarks, demos
- **Community** (16 items, 22%)
  Developer support, newsletter, users meetings, workshops
- **Usability** (12 items, 16%)
  Code readability, ease of use, build system, learning curve, portability
- **Reliability** (4 items, 5%)
  Software quality, unit tests, stable interfaces, few dependencies

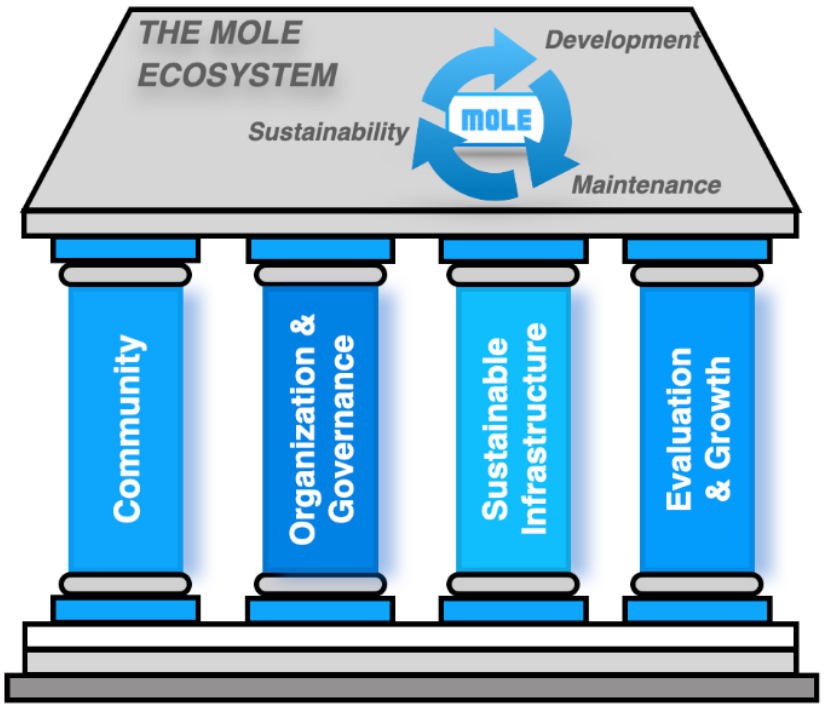
Outline

- MOLE Library and Meeting Introduction (Castillo & Pagallo)
- MOLE OSE Sustainability & Governance (Drummond)
- MOLE Library Updates
  - MATLAB/Octave recent updates (Dumett)
  - C++ recent updates (Drummond)
- Example applications
  - GCCOM II
  - Dispersive Waves
- Summary of MOLE peer-review (Paolini)
- Current and Future MOLE Language Implementations (Paolini & Barra)
- MOLE OSE 2026-2027 Strategic Plans (Castillo, Barra & Drummond)
- Open Floor

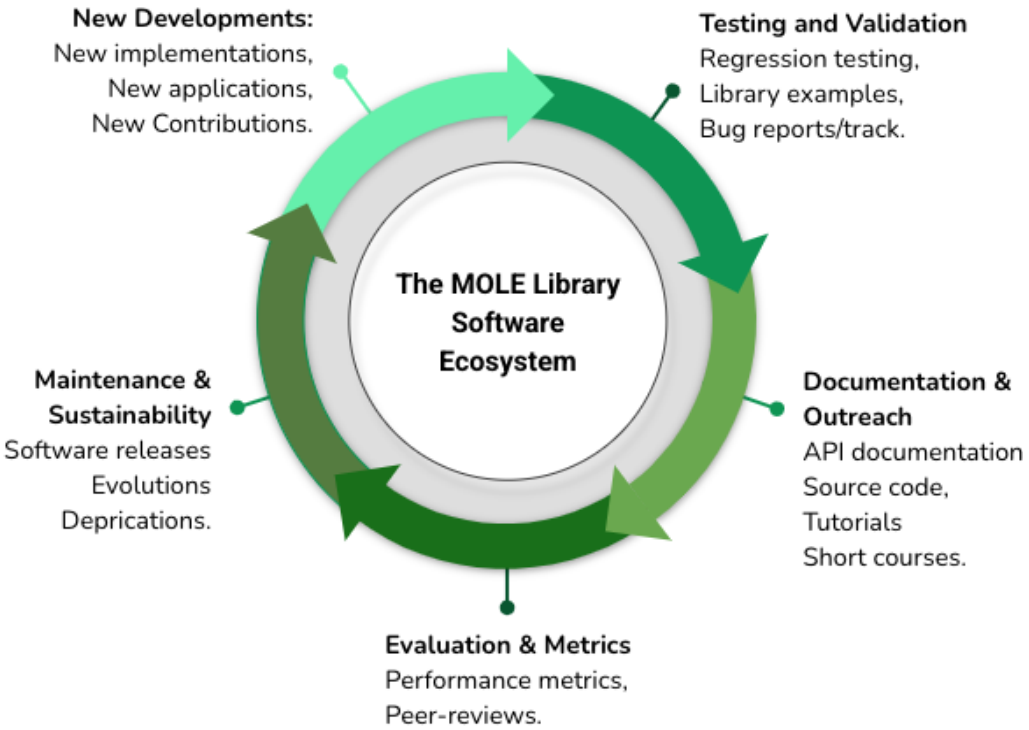
MOLE Open-source Software Ecosystem

The MOLE Open-Source Ecosystem (OSE) organization was established to support the library's

- Sustainable infrastructure,
- Development ecosystem,
- Robustness,
- Reliability,
- Readiness to address a wide-range of computational problems.



MOLE OSE Sustainability



The MOLE OSE supports an interdisciplinary and international community made of:

- New library users
- Advanced library users
- Library developers
- Collaborators
- Sponsors

MOLE OSE Sustainability

Continuous Development Model

- Use GitHub Actions for continuous development, integration (CI), and continuous delivery/deployment (CD),
- Use unit and system tests validate correctness at both the function and API levels, executed automatically,
- Use MATLAB test runner framework support C++ and MATLAB testing,

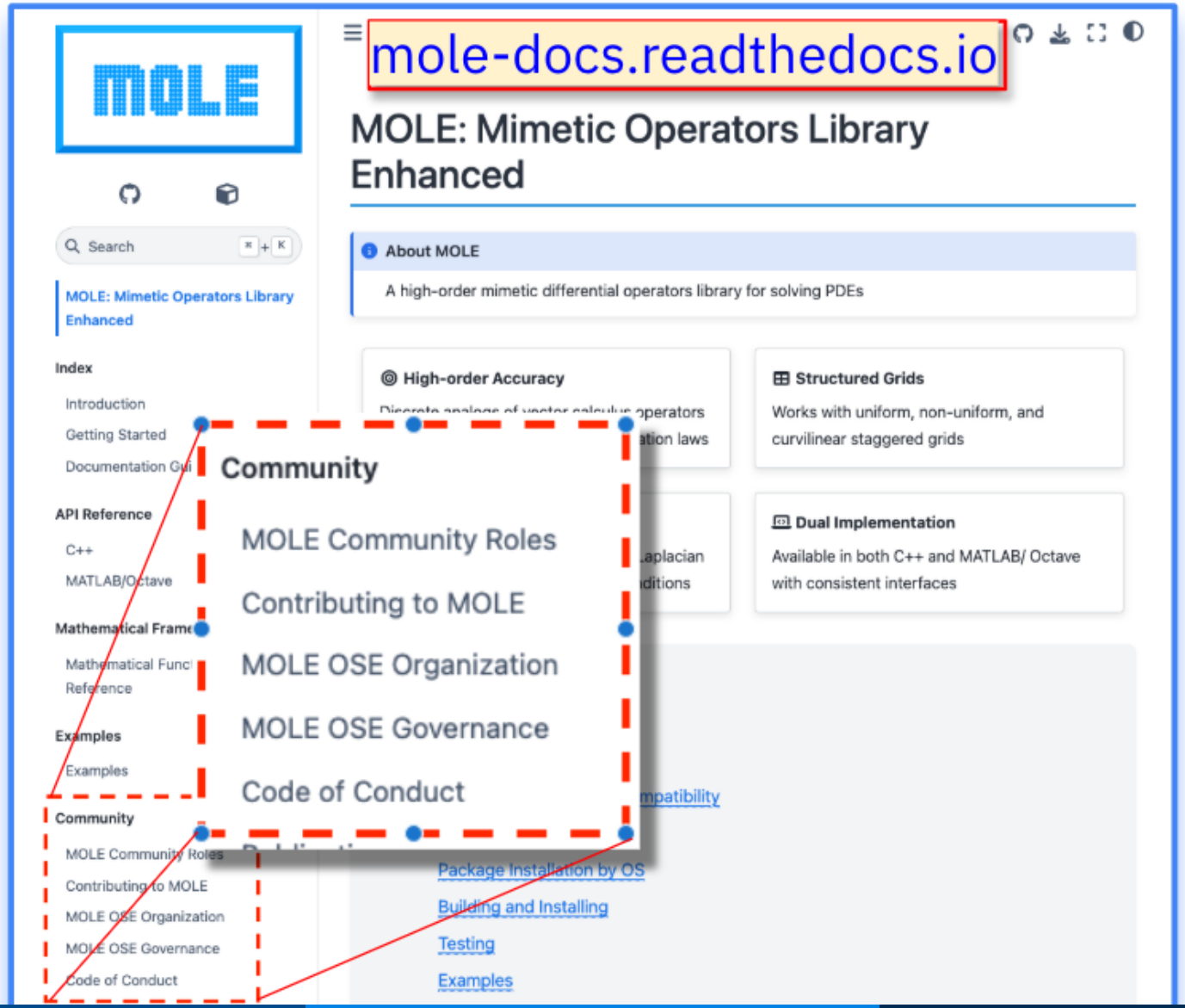
Governance Documents

We encourage everyone's contributions to the MOLE Library. The OSE governance documents support a distributed development environment that safeguards the library's quality and reliability, while encouraging productive and respectful collaborations.

- For new contributors;
  - Contributing to MOLE
  - Code of Conduct
  - Software Design Guidelines
- For everyone in the MOLE OSE community;
  - Community Roles
  - MOLE OSE Organization
  - MOLE OSE Governance

MOLE OSE Governance Documents

Where to find the MOLE OSE Governance Documents?



MOLE Leadership Team

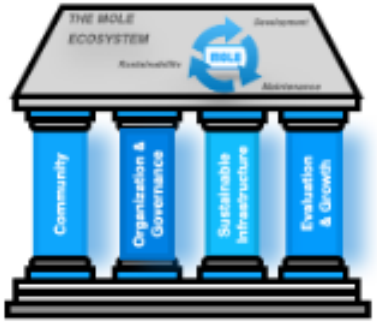
The **MOLE Leadership Team** is made up of the library development founders, current lead developers, and software maintainers and managers of the MOLE OSE.

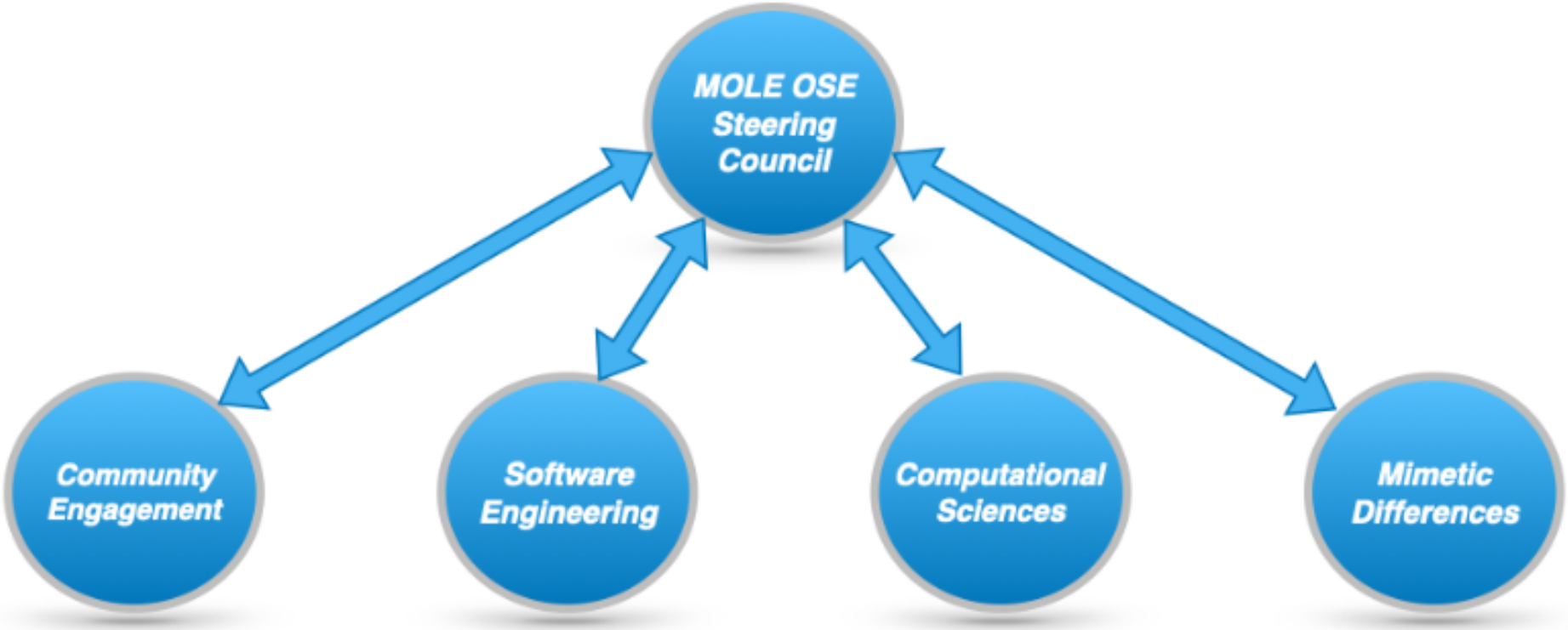
The leadership team is responsible for maintaining:

- Cohesiveness,
- Robustness,
- Security,
- Reliability,
- Succession plans,
- Integrity, and
- Quality of the MOLE library.

This team meets weekly.

MOLE OSE Governance Model





Outline

- MOLE Library and Meeting Introduction (Castillo & Pagallo)
- MOLE OSE Sustainability & Governance (Drummond)
- MOLE Library Updates
  - MATLAB/Octave recent updates (Dumett)
  - C++ recent updates (Drummond)
- Example applications
  - GCCOM II
  - Dispersive Waves
- Summary of MOLE peer-review (Paolini)
- Current and Future MOLE Language Implementations (Paolini & Barra)
- MOLE OSE 2026-2027 Strategic Plans (Castillo, Barra & Drummond)
- Open Floor

Mimetic Differences

1. Approximate vector calculus spatial differential operators (D, G, C, L) that are used to write PDE models.
2. Not only accuracy (symmetries, conserved quantities, fundamental identities, etc.).
3. Use a staggered grid.
4. Operators are constructed to satisfy the extended Gauss' divergence theorem.
5. They reproduce vector calculus identities in the discrete realm.
6. They preserve important quantities for systems of conservation laws.
7. They have been used successfully in many applications.

MOLE Mimetic Difference Operators

The Mimetic Operators Library Enhanced (MOLE) is an implementation of mimetic difference operators.

MOLE operators can be split into:

- Gradient, divergence, Laplacian and Curl
- Quadratures
- Interpolations
- Boundary

MATLAB/Octave MOLE updates

1. General boundary condition functionality that allows to handle different types of boundary conditions (including periodic ones) on each boundary part.
2. Several new interpolation operators.
3. Improvement of some of the differential operators.
4. Several examples and tests that exhibit how to use the new features.

A virtual tutorial will be given on May 12th, 2026 from 10:00 am to noon.

C++ recent updates

The following will be available in the next release of the MOLE Library (v1.2):

- Divergence, Gradient, Laplacian, and Curl operators in functional parity with the MATLAB/Octave implementations,
- General boundary condition functionality,
- New interpolation operators to bring functional parity with MATLAB/Octave,
- Adoption of Clang-tidy linters

Outline

- MOLE Library and Meeting Introduction (Castillo & Pagallo)
- MOLE OSE Sustainability & Governance (Drummond)
- MOLE Library Updates
  - MATLAB/Octave recent updates (Dumett)
  - C++ recent updates (Drummond)
- Example applications
  - GCCOM II
  - Dispersive Waves
- Summary of MOLE peer-review (Paolini)
- Current and Future MOLE Language Implementations (Paolini & Barra)
- MOLE OSE 2026-2027 Strategic Plans (Castillo, Barra & Drummond)
- Open Floor

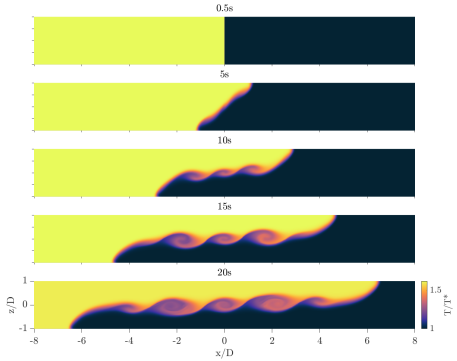
GCCOM II

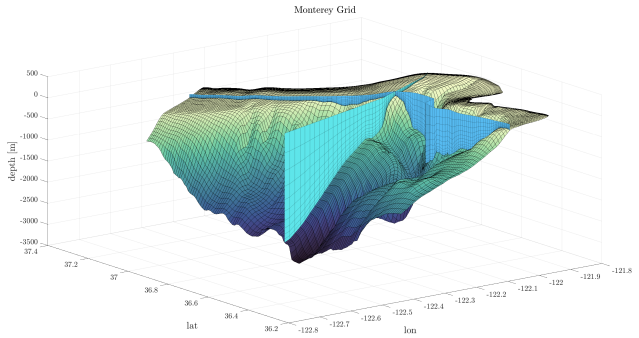
(General Curvilinear Coastal Ocean Model)
Boussinesq non-hydrostatic in C++

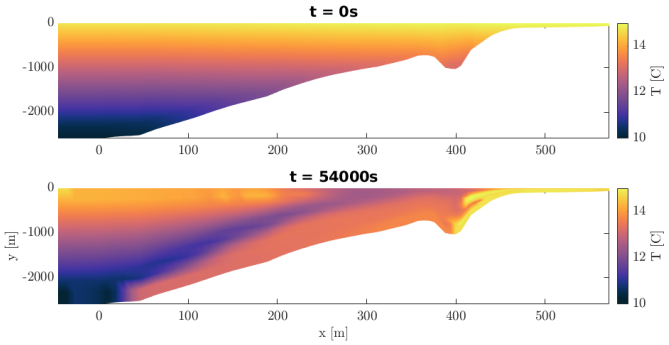
$$\frac{\partial \hat{\boldsymbol{u}}}{\partial \hat{t}} + \hat{\boldsymbol{u}} \cdot (\nabla \hat{\mathbf{u}}) = -\frac{1}{\rho}\nabla \hat{p} + \frac{1}{Re}\nabla^2 \hat{\boldsymbol{u}} + \frac{1}{Fr^2}\hat{S}$$

- MOLE C++ for grad, div, Laplacian
- SuperLU solve for pressure in $Lp = Du^*$
- MOLE C++ for boundary conditions

MOLE for coordinate transform of curvilinear grid

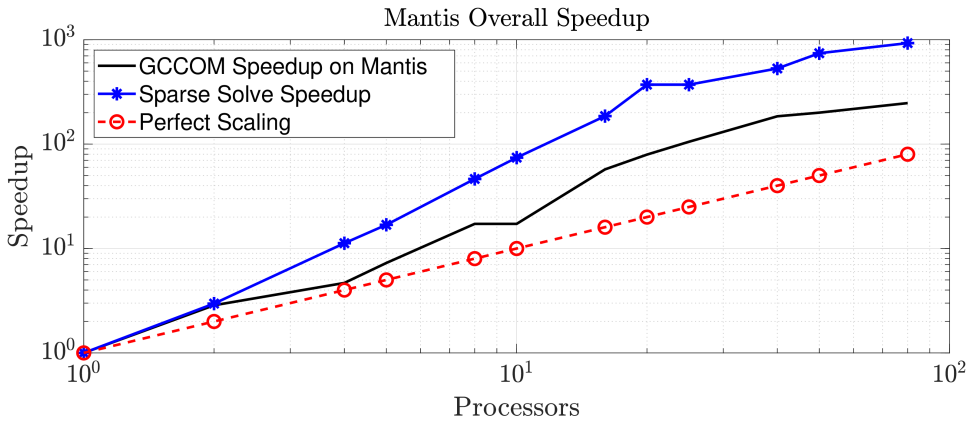


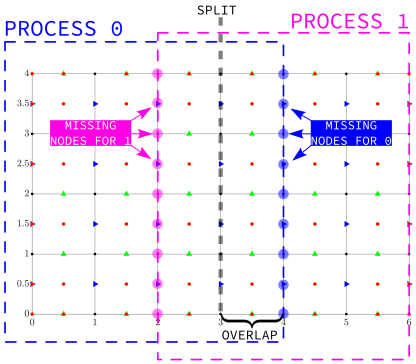


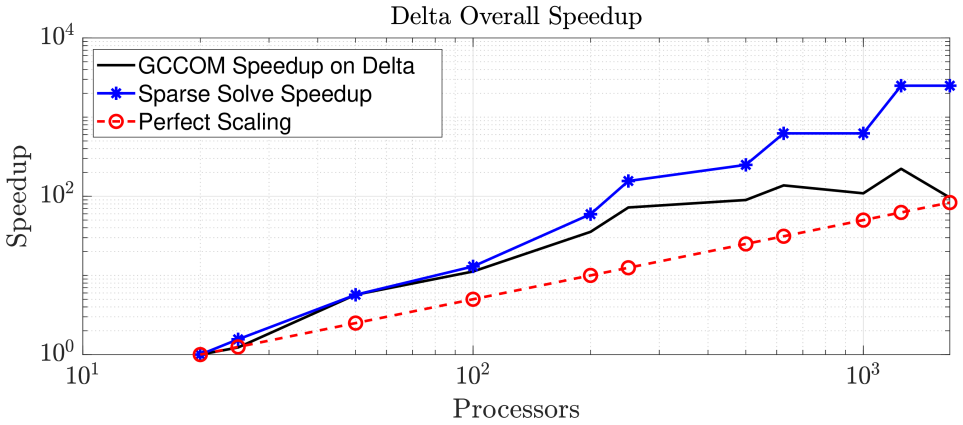


GCCOM II - Parallel

- MOLE BC for interfaces
- SuperLU scaleup rates
- MOLE C++ for boundary conditions







Nonlinear weakly dispersive waves

Outline

- MOLE Library and Meeting Introduction (Castillo & Pagallo)
- MOLE OSE Sustainability & Governance (Drummond)
- MOLE Library Updates
  - MATLAB/Octave recent updates (Dumett)
  - C++ recent updates (Drummond)
- Example applications
  - GCCOM II
  - Dispersive Waves
- Summary of MOLE peer-review (Paolini)
- Current and Future MOLE Language Implementations (Paolini & Barra)
- MOLE OSE 2026-2027 Strategic Plans (Castillo, Barra & Drummond)
- Open Floor

Summary of MOLE peer-review

Respondents: 8 persons and 5 classes

#	Class
7	Numerical Software Developers
4	MOLE Library Users
3	MOLE Library Contributors
2	Software Engineers
1	Theoretical Physicist

Summary of MOLE peer-review

Respondents: Programming Languages Used

#	Language
6	Fortran
6	MATLAB/Octave
3	Python
3	C/C++/ObjectiveC
2	Julia

Summary of MOLE peer-review

Respondents:
Operating Systems
Used

#	O/S
6	macOS
2	Linux
2	Exascale Cluster (e.g., TOSS, HPE Cray)
1	Windows

Summary of MOLE peer-review

Respondents:
Satisfaction with the MOLE GitHub Repository

#	Rank
6	Excellent
1	Very good
0	Good
0	Poor
0	Inadequate
1	Decline to answer

Summary of MOLE peer-review

Respondents:
Satisfaction with the
MOLE
MATLAB/Octave API

#	Rank
5	Excellent
1	Very good
0	Good
0	Poor
0	Inadequate
2	Decline to answer

Summary of MOLE peer-review

Respondents:
Satisfaction with the
MOLE
MATLAB/Octave
Documentation

#	Rank
2	Excellent
3	Very good
1	Good
0	Poor
0	Inadequate
2	Decline to answer

Summary of MOLE peer-review

Respondents:
Satisfaction with the
MOLE C++ API

#	Rank
0	Excellent
3	Very good
0	Good
0	Poor
0	Inadequate
5	Decline to answer

Summary of MOLE peer-review

Respondents:
Satisfaction with the MOLE C++ Documentation

#	Rank
0	Excellent
2	Very good
1	Good
0	Poor
0	Inadequate
5	Decline to answer

Summary of MOLE peer-review

Respondents:
Satisfaction with the
MOLE Build System

#	Rank
1	Excellent
4	Very good
0	Good
0	Poor
0	Inadequate
3	Decline to answer

Summary of MOLE peer-review

Respondents:
Satisfaction with the MOLE Automatic Regression Testing System

#	Rank
3	Excellent
1	Very good
2	Good
0	Poor
0	Inadequate
2	Decline to answer

Summary of MOLE peer-review

Respondents:
Satisfaction with the MOLE Onboarding process for Developers

#	Rank
3	Excellent
2	Very good
0	Good
0	Poor
0	Inadequate
3	Decline to answer

Summary of MOLE peer-review

Respondents:
Satisfaction with the *User Friendliness* of MOLE

#	Rank
3	Excellent
2	Very good
1	Good
0	Poor
0	Inadequate
2	Decline to answer

Summary of MOLE peer-review

Respondents:
Selected Comments

- MOLE is a highly intuitive and efficient library. Its open-source nature and the direct link between theory and code are its greatest strengths. I look forward to seeing the project grow, and I hope to contribute to the repository soon to help expand the library's capabilities.
- Thanks a lot for investing time in developing MOLE :)
- I've noticed significant updates to the documentation since I started looking. It's coming together.
- Great group, and ambitious!

Outline

- MOLE Library and Meeting Introduction (Castillo & Pagallo)
- MOLE OSE Sustainability & Governance (Drummond)
- MOLE Library Updates
  - MATLAB/Octave recent updates (Dumett)
  - C++ recent updates (Drummond)
- Example applications
  - GCCOM II
  - Dispersive Waves
- Summary of MOLE peer-review (Paolini)
- Current and Future MOLE Language Implementations (Paolini & Barra)
- MOLE OSE 2026-2027 Strategic Plans (Castillo, Barra & Drummond)
- Open Floor

MOLE Language Implementations

The following are full language implementations of the MOLE library:

- MATLAB/Octave
- C++
- Python (PyMOLE)
- Julia (MOLE.jl)
- Fortran (Formal developed by Dr. Damian Rouson at LBNL)

Python for Scientific Computing

- Python has emerged as a standard generalist language for science and engineering applications and a competitor to MATLAB.
- Python enables development at the same level of abstraction as MATLAB, allowing the programmer to focus on mathematics and algorithms rather than memory management and low-level data structures.
- The NumPy (i.e., *Numerical Python*) core library facilitates complex operations on N-dimensional arrays (vectors, matrices, tensors), which is especially relevant for implementing mimetic operators and solvers.
- The SciPy core library provides support for decompositions (LU) and sparse matrix handling for solving large systems of equations.

Existing Implementations

- `https://github.com/Dilip-Jain/pymole`
  - Dilip Jain, `medilipjain@gmail.com`, Machine Learning Engineer at Infosys, Pune District, Maharashtra, India.
  - Last commit: 3 months ago
- `https://github.com/nutrik/pymole`
  - Roman Nuterman, `nuterman@nbi.ku.dk`, Research Coordinator in Physical Oceanography at the Niels Bohr Institute - University of Copenhagen, Denmark
  - Last commit: 5 years ago
  - Not compatible with Numpy > 1.20
- `https://github.com/carlosal1015/pymole`
  - Carlos Alonso Aznarán Laos, caznaranl@uni.pe, student at the National University of Engineering (UNI) in Lima, Peru.
  - Last commit: 3 months ago

Carlos' pytest Report

```
PS C:\Users\paoli\Documents\GitHub\carlos_pymole> python3.13.exe -m pytest
========================================================= test session starts =========================================================
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\paoli\Documents\GitHub\carlos_pymole
configfile: pyproject.toml
testpaths: tests, notebooks
plugins: nbval-0.11.0, cov-7.1.0
collected 12 items

tests\test_divergence.py ....                                                                                                    [ 33%]
tests\test_gradient.py ....                                                                                                      [ 66%]
tests\test_laplacian.py ...                                                                                                      [ 91%]
tests\test_pymole.py .                                                                                                           [100%]

========================================================= warnings summary ============================================================
tests/test_divergence.py: 4 warnings
tests/test_gradient.py: 4 warnings
tests/test_laplacian.py: 3 warnings
  C:\Users\paoli\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313
\site-packages\scipy\sparse\_index.py:210: SparseEfficiencyWarning: Changing the sparsity structure of a csr_matrix is expensi
ve. lil and dok are more efficient.
    self._set_arrayXarray(i, j, x)

tests/test_divergence.py: 3 warnings
tests/test_gradient.py: 4 warnings
tests/test_laplacian.py: 3 warnings
  C:\Users\paoli\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313
\site-packages\scipy\sparse\_index.py:201: SparseEfficiencyWarning: Changing the sparsity structure of a csr_matrix is expensi
ve. lil and dok are more efficient.
    self._set_arrayXarray_sparse(i, j, x)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================================= 12 passed, 21 warnings in 8.13s =================================================
```

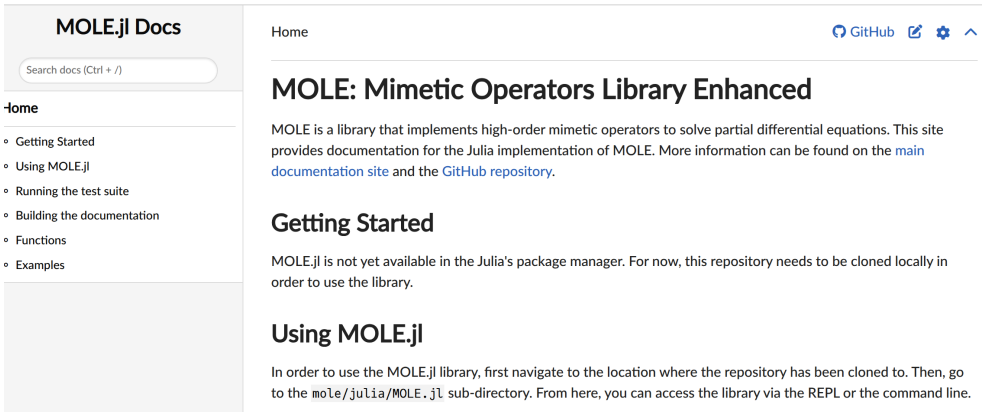
Carlos' PyMOLE Support

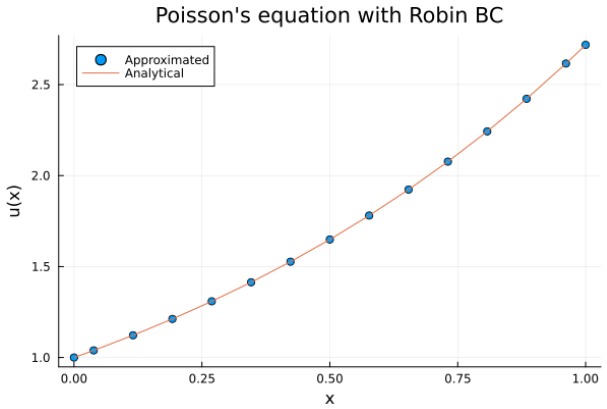
- divergence: div1DNonUniform.py div1D.py div2DNonUniform.py div2D.py div3DNonUniform.py div3D.py
- gradient: grad1DNonUniform.py grad1D.py grad2DNonUniform.py grad2D.py grad3DNonUniform.py grad3D.py
- laplacian: lap1D.py lap2D.py lap3D.py
- interpolation: interpol1D.py interpol2D.py interpol3D.py interpolD1D.py interpolD2D.py interpolD3D.py
- boundary conditions: robinBC1D.py robinBC2D.py robinBC3D.py

Why Julia?

- Julia is an innovative new(er) open-source programming language for high-level, high-performance numerical computing
- Julia combines the ease-of-use, numeric focus, and graphics of Matlab, the speed of C and Fortran, the general-purpose breadth, and extensibility of Python
- Julia's package manager facilitates reproducibility in scientific computing, allowing for easy sharing of package environments (cross-platforms)
- Julia uses type inference and just-in-time (JIT) compilation to compile high-level user code to machine code on the fly
  - As a result, Julia is competitive with Matlab for interactive exploration and with C and Fortran for high-performance computing

Current MOLE.jl developments





- Started as student contributions supervised/reviewed by faculty: Marc, Ben, Jose
  - 1D operators (grad, div, Lap), BCs, and examples: elliptic1D.jl
- Current/recent developments:
  - package modularization
  - 1D interpolation operator and examples: hyperbolic1D.jl
  - 2D (grad, div, Lap), BCs, and elliptic and parabolic examples
  - improved documentation
- Outlook:
  - Presenting a poster at JuliaCon Global 2026 conference in Mainz, Germany

Outline

- MOLE Library and Meeting Introduction (Castillo & Pagallo)
- MOLE OSE Sustainability & Governance (Drummond)
- MOLE Library Updates
  - MATLAB/Octave recent updates (Dumett)
  - C++ recent updates (Drummond)
- Example applications
  - GCCOM II
  - Dispersive Waves
- Summary of MOLE peer-review (Paolini)
- Current and Future MOLE Language Implementations (Paolini & Barra)
- MOLE OSE 2026-2027 Strategic Plans (Castillo, Barra & Drummond)
- Open Floor

MOLE 2026-2027 Strategic Plans

MOLE Library Enhancements:

- Expand more general boundary condition functionalities for PDEs (e.g., curvilinear geometries and more robust curl operators),
- We continue to bring MOLE's C++ functionality to par with the MATLAB/Octave implementation — *we need to expand the number of C++ examples.*
- Formalize methods to evaluate the effectiveness of MOLE's numerical algorithms, focusing on accuracy, algorithmic complexity, and time to solution across computational platforms,
- We need to provide software utilities for the coupling of multiphysics models,
- Implement error propagation and tracing capabilities into the MOLE library to support resilient integrations with third-party software.

MOLE 2026-2027 Strategic Plans

Risk Analysis and Security Plan

- Continue to work on critical security focus areas for MOLE, Currently, the focus areas are:
  - GitHub source code repository, and
  - Packaged software distributions,
- Address Software vulnerabilities
- Planning on submitting a proposal for NSF Safety, Security, and Privacy of Open-Source Ecosystems (Safe-OSE) program

MOLE 2026-2027 Strategic Plans

Community Engagement

- Increasing awareness and interest in the MOLE library as well as in recruiting new developers,
- Need to work with existing MOLE-user based on exploring other areas of applicability.
- Create short courses and tutorials,
- Publish MOLE related work,
- Track publications from MOLE Users, and
- Send periodic MOLE related updates through mailing lists.

MOLE 2026-2027 Strategic Plans

Identifying, Engage, and Support Users And Partners:

- Create more hands-on tutorials,
- Maintain and periodically review the governance documents to support all MOLE stakeholders.
- Maintain and review web-based onboarding documentation with a clear code of conduct, contributing guidelines, submission templates, and an authorship policy to provide credit to contributors.
- Recruit peer-reviewers for the MOLE OSE. We welcome input from the MOLE community (through mole-ose.org or MOLE's Github repository).

A thriving community is essential to MOLE's success and sustainability.

MOLE OSE Sustainability

- Target new areas of applicability,
- Coordinate with stakeholders to maintain succession plans for both internal and external development of the MOLE library,
- Use of metrics to evaluate the effectiveness of the current releases of the MOLE library, these are based on software functionalities and corresponding documentation,
- Collaborate with the Extreme-scale Scientific Software Stack (E4S) collection,
- Continue to receive and encourage contributions from an engaged user community

MOLE OSE Financial Sustainability

- Continue to apply for funding at national funding agencies,
- Explore commercial sources of funding through strategic collaborations and industrial partnerships.

Outline

- MOLE Library and Meeting Introduction (Castillo & Pagallo)
- MOLE OSE Sustainability & Governance (Drummond)
- MOLE Library Updates
  - MATLAB/Octave recent updates (Dumett)
  - C++ recent updates (Drummond)
- Example applications
  - GCCOM II
  - Dispersive Waves
- Summary of MOLE peer-review (Paolini)
- Current and Future MOLE Language Implementations (Paolini & Barra)
- MOLE OSE 2026-2027 Strategic Plans (Castillo, Barra & Drummond)
- Open Floor

THANK YOU

We acknowledge the ever-growing **MOLE Community**:

1. *Contributors:* Valeria Barra, Angel Boada, Jared Brzenski, Jose Castillo, Prit Chakalasiya, Surinder Chhabra Singh, Johnny Delgado Corbino, Tony Drummond, Miguel Dumett, Joe Hellmers, Arshia Ilaty, Katayoon Kaviani, Oluchi Nzerem, Giulia Pagallo, Christopher Paolini, Jiya Rathi, Valentina Rosano, Aneesh Murthy Srinivas, Janani Priyadharshini Srinivasan, Manuel Valera, Ben Wagner, Deepanshu Zade, Jose Baranda, Ryan Caywood, Marc Loschen, Jordan Shands-Sparks
2. *MOLE Library 2026 Peer-reviewers:* David Kopriva, Joe Hellmers, Christos Papoutsellis, Antonio Redondo, Damian Rouson, Catalina Maria Rúa Alvarez, Christian Schaerer, Jhabriel Varela
3. All collaborators and users
4. Sponsors: NSF and SDSU CSRC

MUG Meeting Open Floor

We would like to receive your feedback on the MOLE library and OSE. Here are few ways in which you can reach us:

1. During the live MUG event, please raise a hand (online and in person participants)
2. Visit the mole-ose.org website, under *Events*, you will find a link to the MUG 2026 meetings where you can:
   - Provide a peer-review of the library
   - Send us feedback on the MOLE Library or the MOLE OSE
   - For software related features and issues, please visit the MOLE Github repository: github.com/csrc-sdsu/mole and submit an issue or start a GitHub conversation.